

AMA 564 Deep Learning

2026 Spring

Lecture 8

Generative Adversarial Networks (GAN)

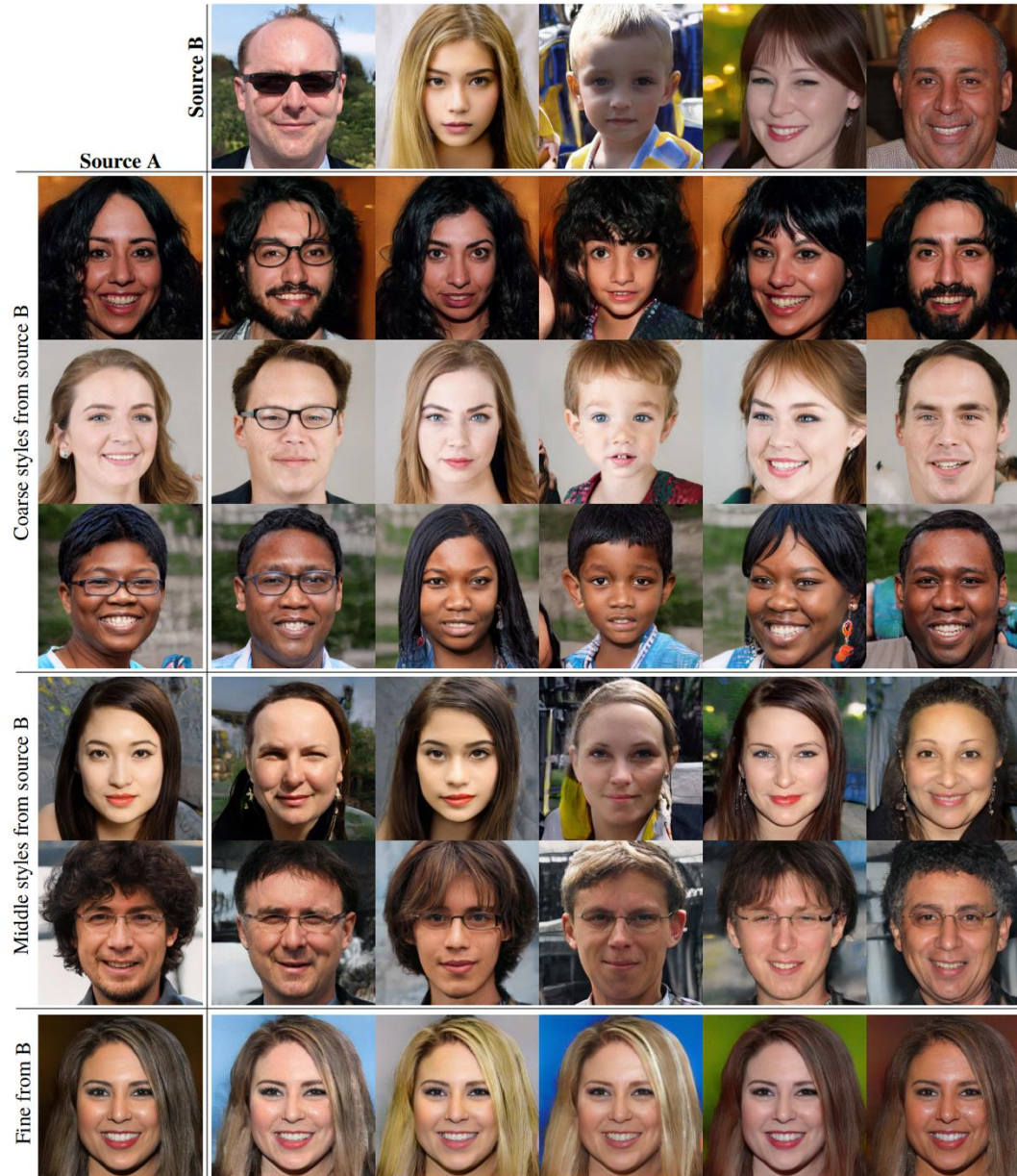
Demo of GANs



Example of Photorealistic Human Faces Generated by a GAN.

Source: <https://arxiv.org/abs/1812.04948>

Demo of GANs



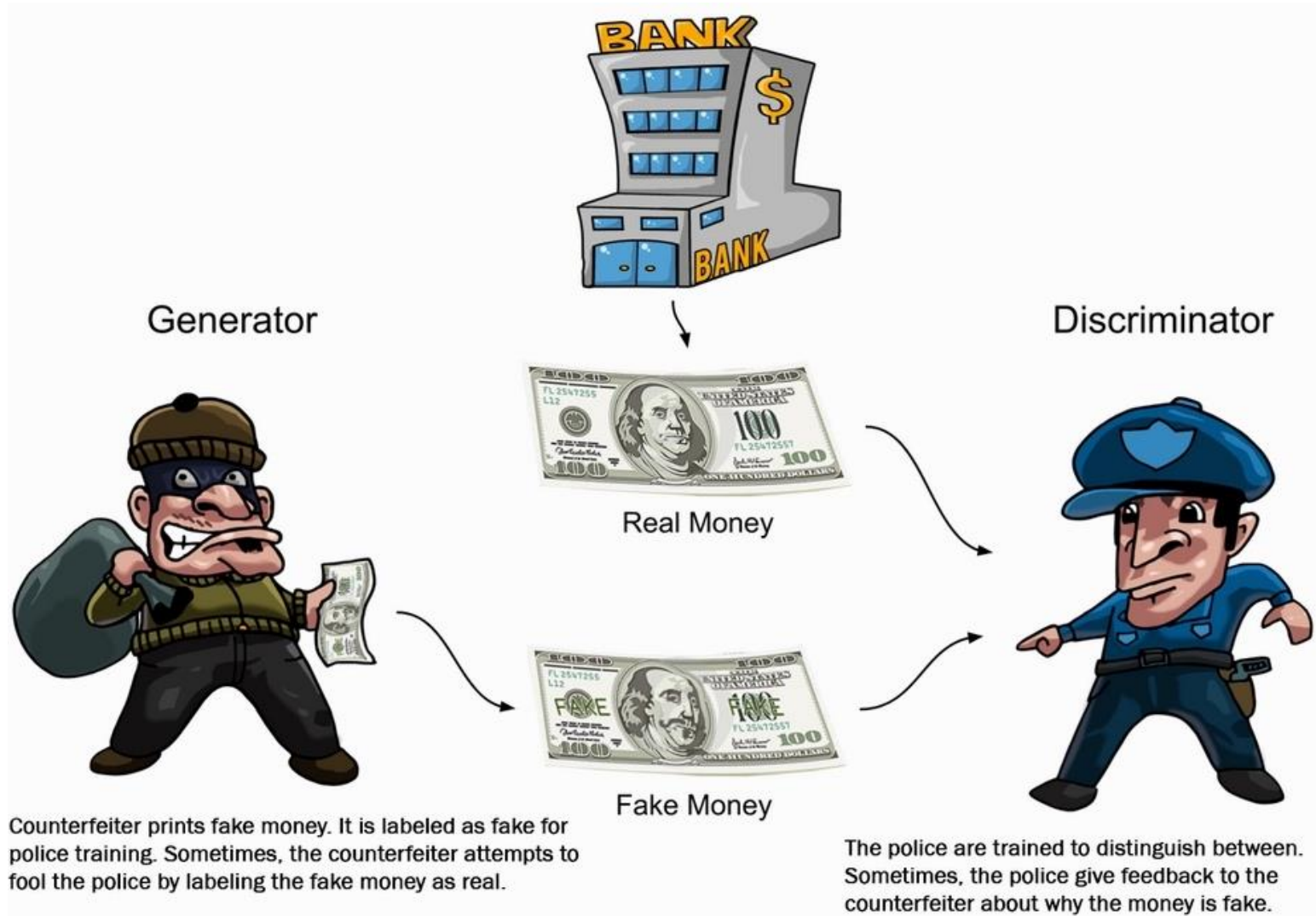
Source: <https://arxiv.org/abs/1812.04948>

*In the proposed adversarial nets framework, the generative model is pitted against an adversary: a discriminative model that learns to determine whether a sample is from the model distribution or the data distribution. **The generative model can be thought of as analogous to a team of counterfeiters, trying to produce fake currency and use it without detection,** while the discriminative model is analogous to the police, trying to detect the counterfeit currency. Competition in this game drives both teams to improve their methods until the counterfeits are indistinguishable from the genuine articles.*

Reference:

Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., ... & Bengio, Y. (2020). **Generative adversarial networks**. *Communications of the ACM*, 63(11), 139-144. **Citation: 56,388**

What is GAN?



Competition in this game drives both teams to improve their methods until the counterfeits are indistinguishable from the genuine articles.

What is GAN?



Generated Data

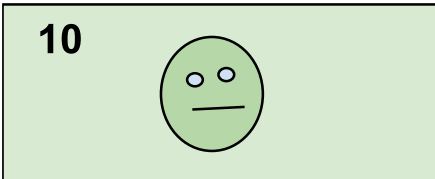
Discriminator

Real Data



FAKE

REAL



FAKE

REAL

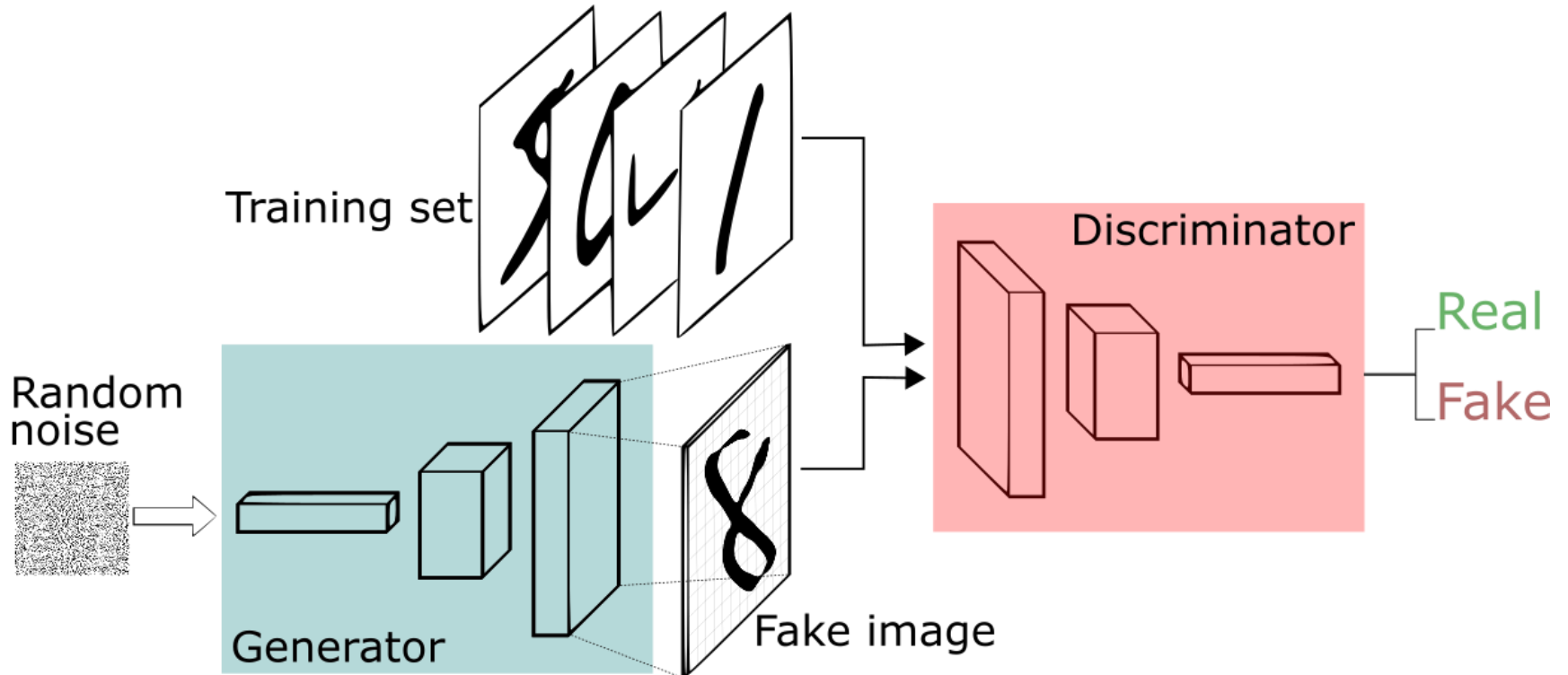


REAL

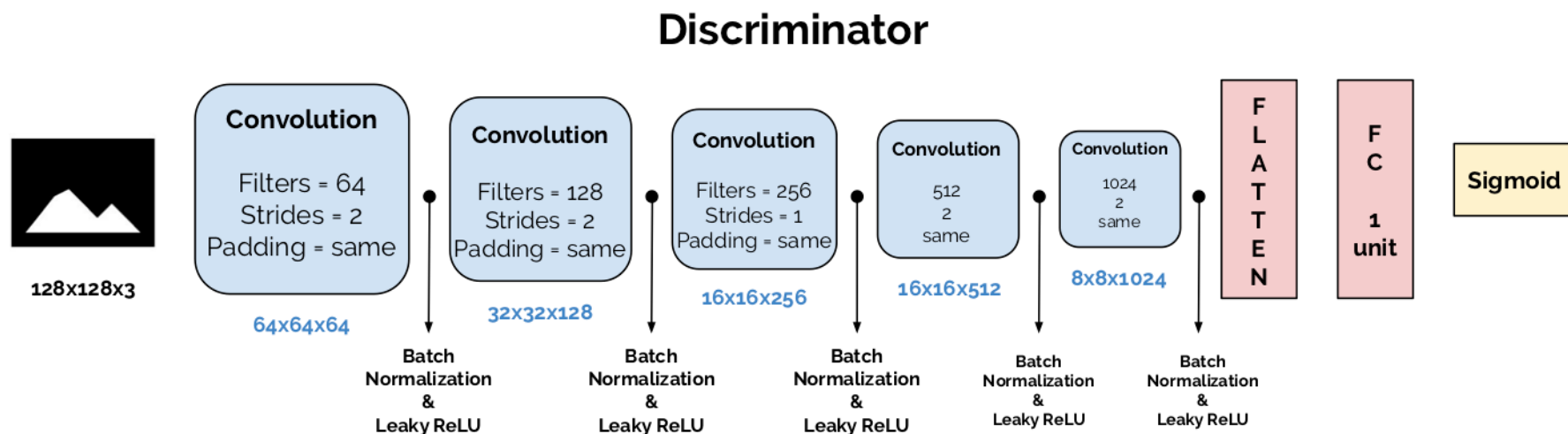
REAL



The architecture of GAN



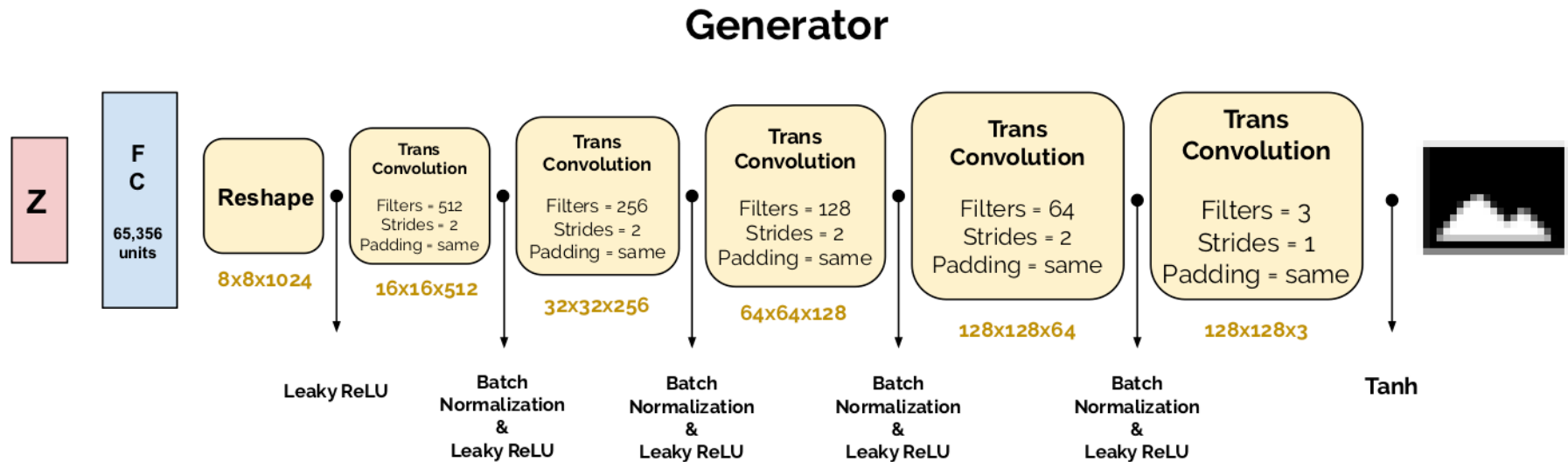
The architecture of Discriminator



D: Discriminator, a neural network, $D(x) \in [0, 1]$

Goal: $D(x) = 1$ if x comes from the **real data**
 $D(x) = 0$ if x comes from the **generated data**

The architecture of Generator



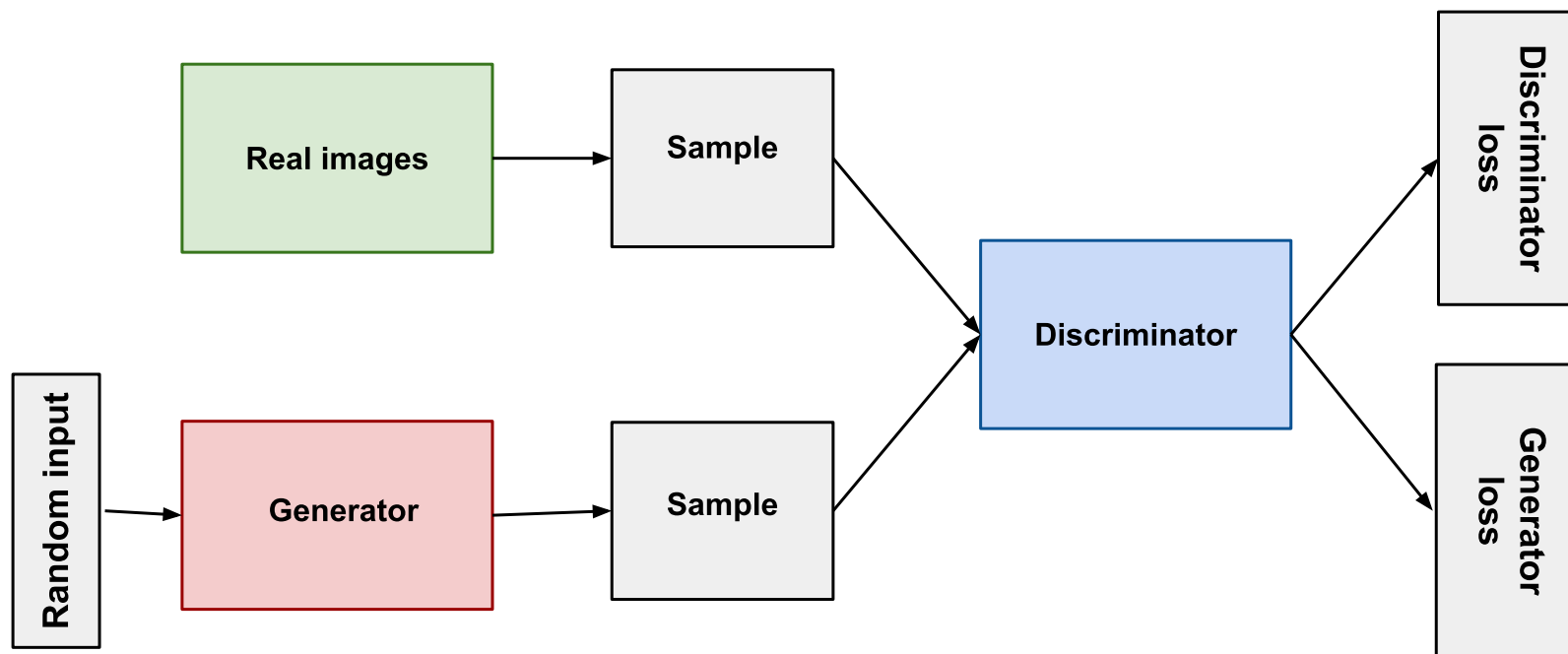
G : Generator, a neural network, $G(z) = x_z$

Goal: z comes from a noise distribution, $G(z) = x_z$

x comes from the real data distribution

Hope $G(z) = x_z$ have the same distribution as x such that discriminator can not tell the difference between them

The workflow of GAN



The value function of GAN

$$V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))]$$

p_{data} the density of data distribution

$p_{\mathbf{z}}$ the density of noise distribution

D Discriminator, a neural network
Maximize $V(D, G)$

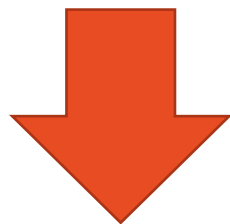
G Generator, a neural network
minimize $V(D, G)$

Target of discriminator

$$V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))]$$

$D(\mathbf{x}) \rightarrow 1$ for $\mathbf{x}$ comes from the **real data**

$D(G(\mathbf{z})) \rightarrow 0$ for $G(\mathbf{z})$ comes from the **generated data**

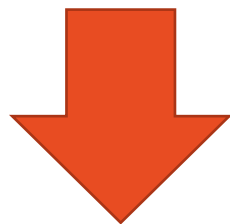


Given generator G
the value $V(D, G)$ **increases**

Target of generator

$$V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))]$$

Generate $G(\mathbf{z})$ where $\mathbf{z}$ comes from the **noise distribution**
such that $D(G(\mathbf{z})) \rightarrow 1$



Given discriminator D
the value $V(D, G)$ **decreases**

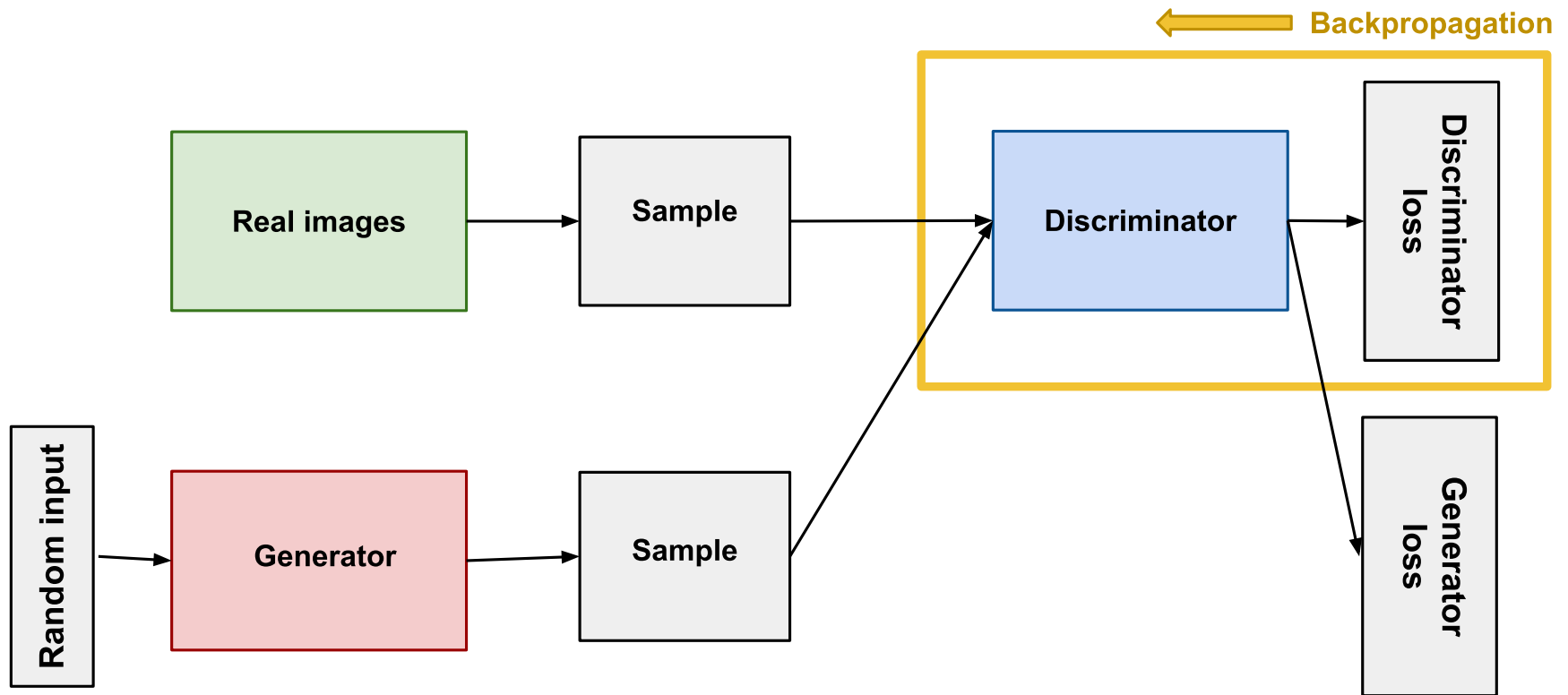
The objective of GAN: Two-player minimax game

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))]$$

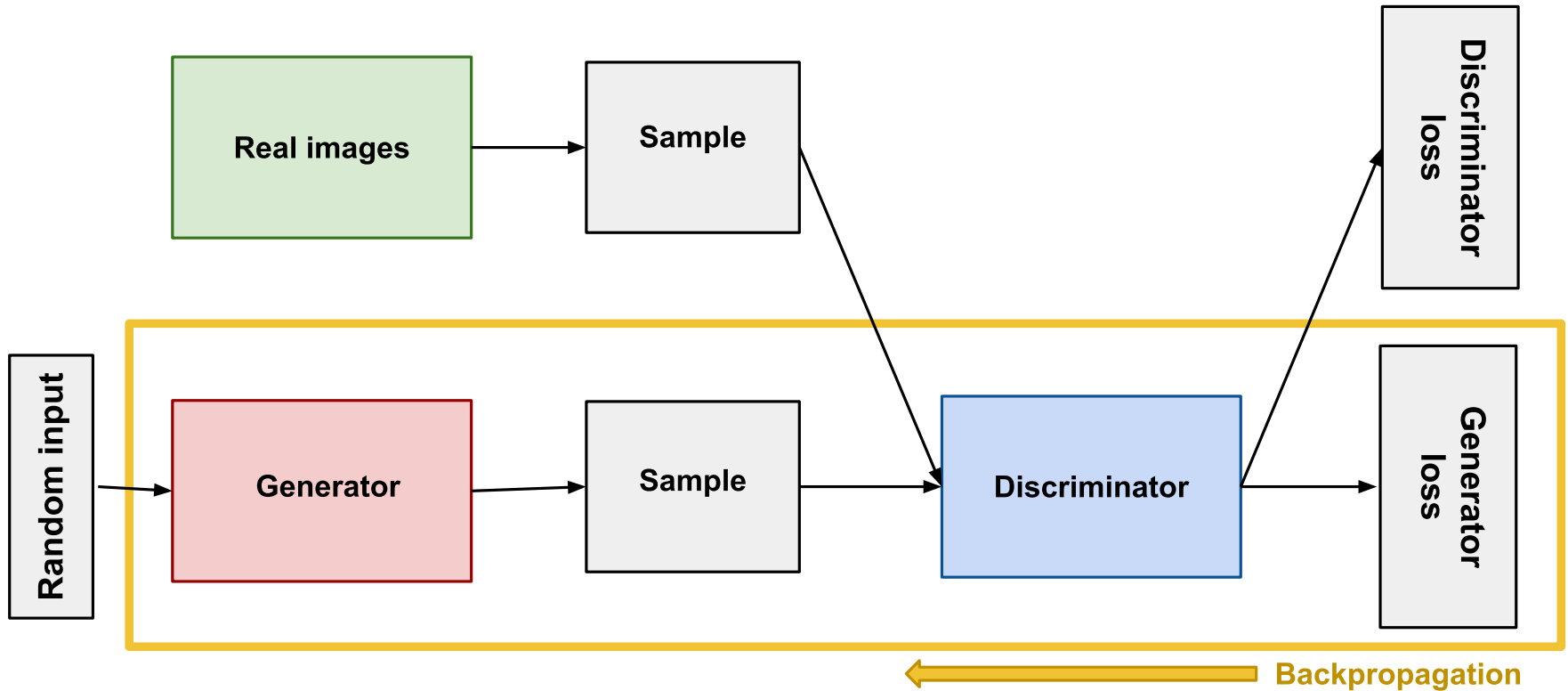
D: Discriminator, a neural network
maximize $V(D, G)$

G: Generator, a neural network
minimize $V(D, G)$

The training of discriminator



The training of generator



Algorithm 1 Minibatch stochastic gradient descent training of generative adversarial nets. The number of steps to apply to the discriminator, k , is a hyperparameter. We used $k = 1$, the least expensive option, in our experiments.

for number of training iterations **do**

for k steps **do**

- Sample minibatch of m noise samples $\{\mathbf{z}^{(1)}, \dots, \mathbf{z}^{(m)}\}$ from noise prior $p_g(\mathbf{z})$.
- Sample minibatch of m examples $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ from data generating distribution $p_{\text{data}}(\mathbf{x})$.
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[\log D(\mathbf{x}^{(i)}) + \log \left(1 - D(G(\mathbf{z}^{(i)})) \right) \right].$$

end for

- Sample minibatch of m noise samples $\{\mathbf{z}^{(1)}, \dots, \mathbf{z}^{(m)}\}$ from noise prior $p_g(\mathbf{z})$.
- Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log \left(1 - D(G(\mathbf{z}^{(i)})) \right).$$

end for

The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.



Proposition 1. *For G fixed, the optimal discriminator D is*

$$D_G^*(\mathbf{x}) = \frac{p_{data}(\mathbf{x})}{p_{data}(\mathbf{x}) + p_g(\mathbf{x})}$$

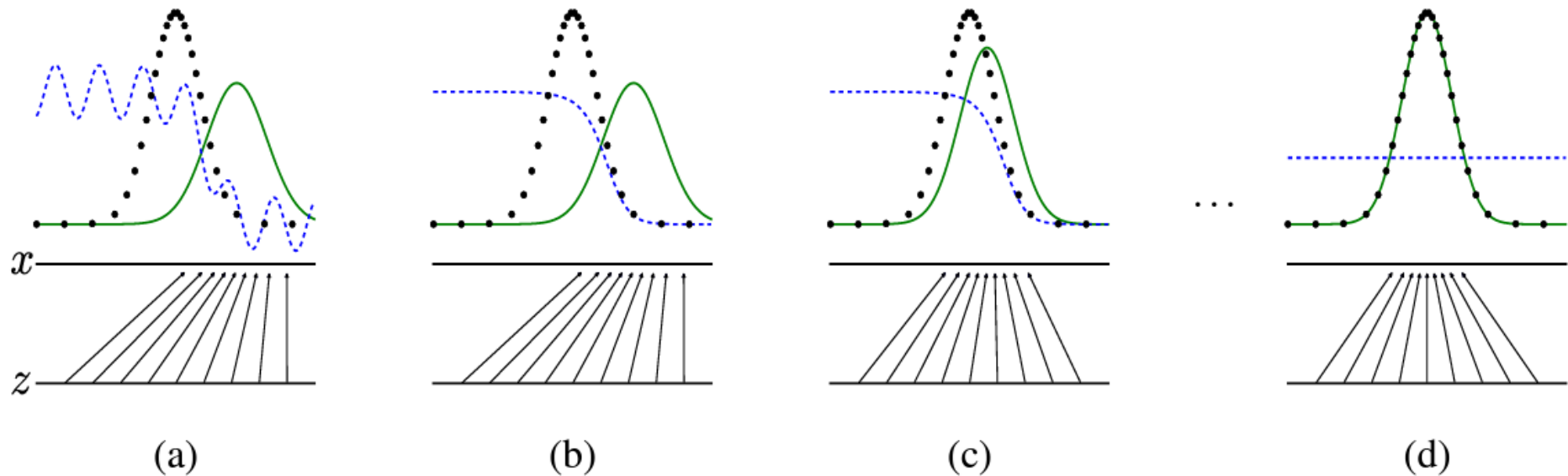
Proposition 2. *If G and D have enough capacity, and at each step of Algorithm 1, the discriminator is allowed to reach its optimum given G , and p_g is updated so as to improve the criterion*

$$\mathbb{E}_{\mathbf{x} \sim p_{data}} [\log D_G^*(\mathbf{x})] + \mathbb{E}_{\mathbf{x} \sim p_g} [\log(1 - D_G^*(\mathbf{x}))]$$

then p_g converges to p_{data}

- $G^*(Z)$ has the same distribution as X
- $D^*(x) = 1/2$ for all x

Train the discriminator and generator alternatively.



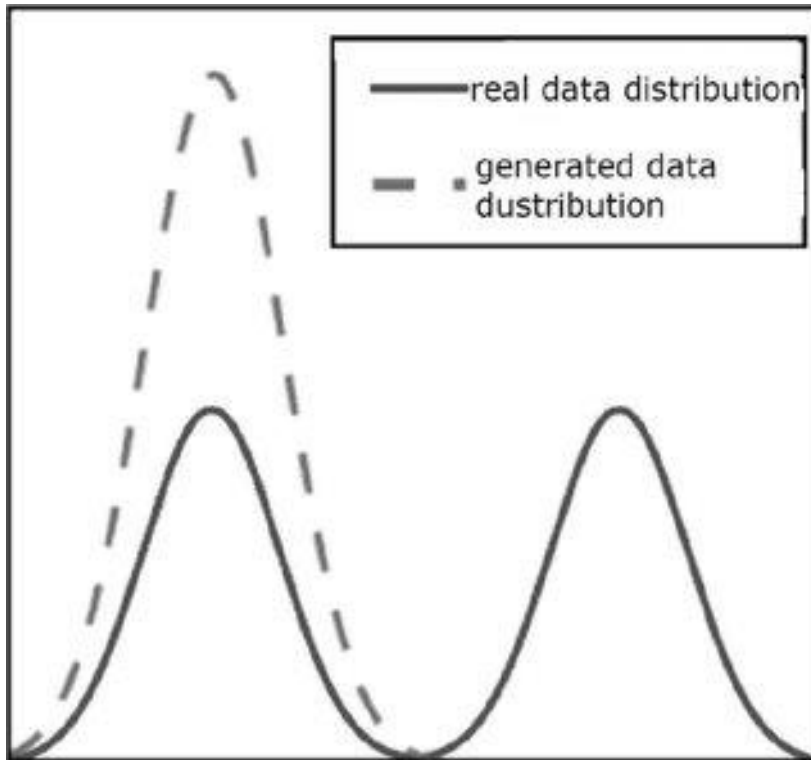
Discriminative distribution:
Generative distribution:
Data distribution:

blue, dashed line
green, solid line
black, dotted line

Difficulties in training GAN

- **Model doesn't converge**
 - G,D parameters may oscillate
 - Uneven progress between G,D
- **Mode collapse (the Helvetica scenario)**
 - G will only generate samples from a single mode
- **Samples lack global structure**
 - E.g., Some generated faces will have 3 eyes

Mode collapse



(a)



(b)

Source: https://www.researchgate.net/publication/354203725_Modified_generative_adversarial_networks_for_image_classification

Data sets: *MNIST*; *the Toronto Face Database (TFD)*; *CIFAR-10*



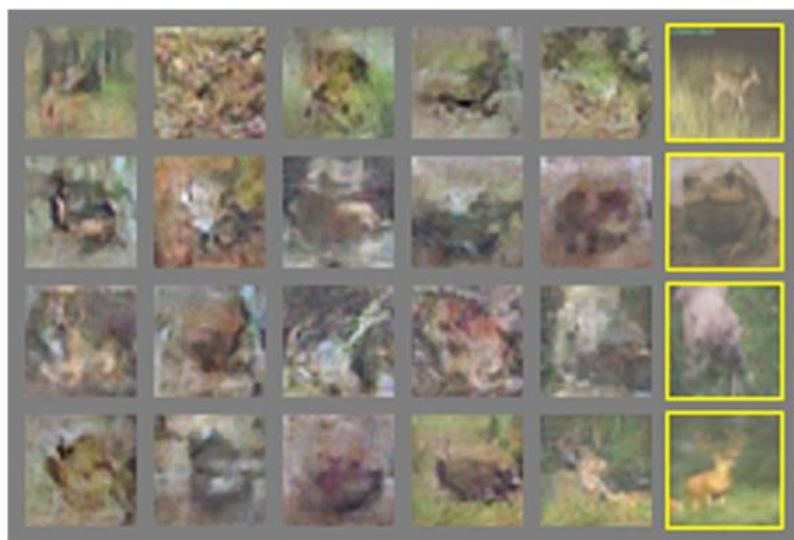
a)



b)



c)



d)

Evolution of GAN



2014



2015



2016



2017



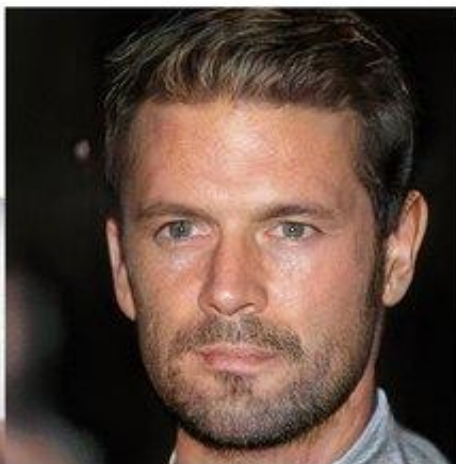
2014
GANs



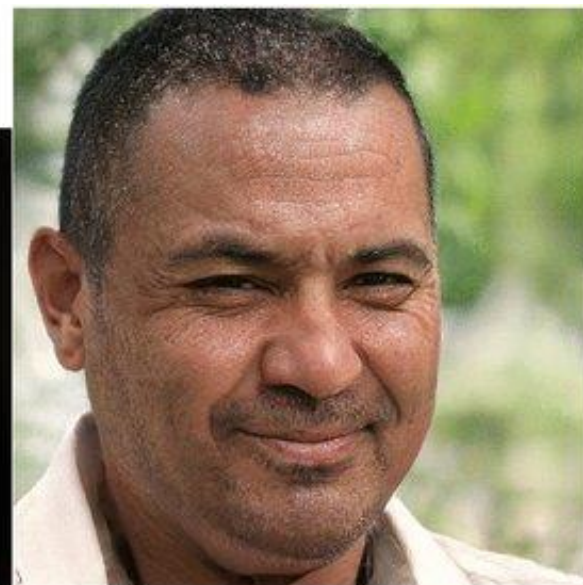
2015
DCGAN



2016
CoGAN



2017
Progressive
Growing of GANs

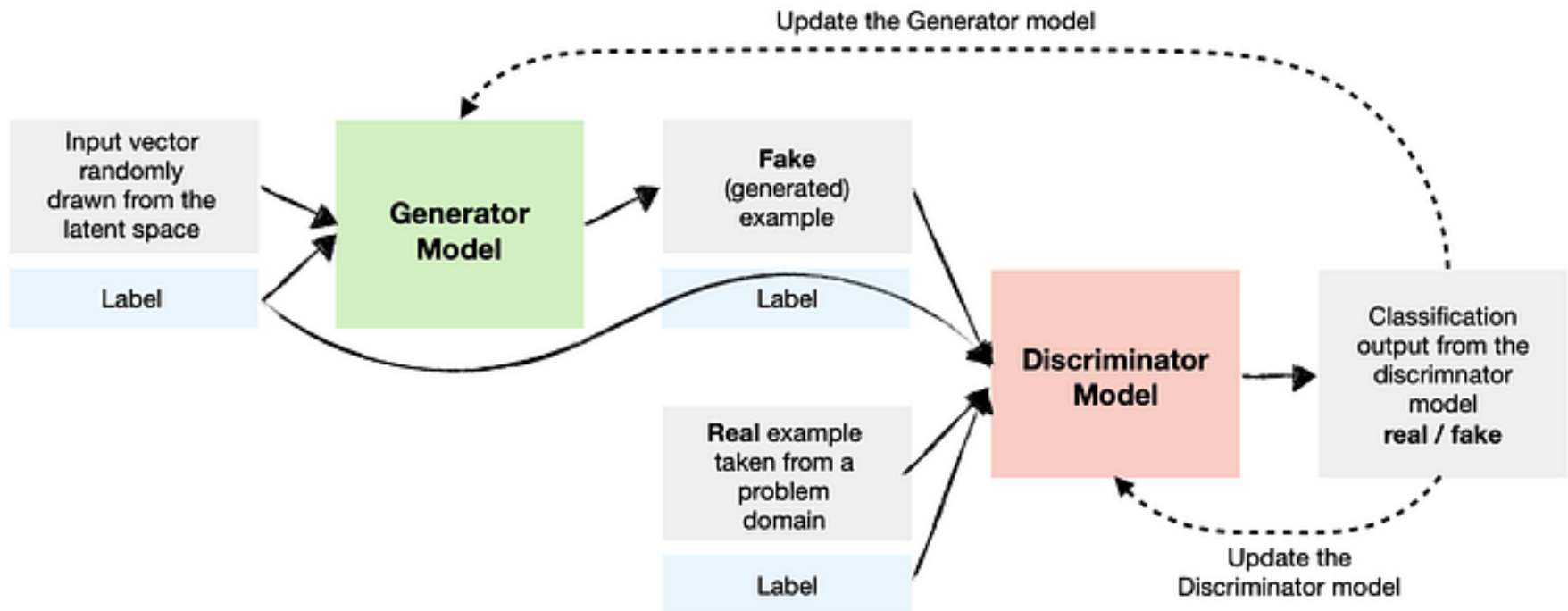


2018
StyleGAN

Popular GAN models and applications

Reference:

Mirza, M., & Osindero, S. (2014). **Conditional generative adversarial nets**. *arXiv preprint arXiv:1411.1784*



Source: <https://solclover.com/>

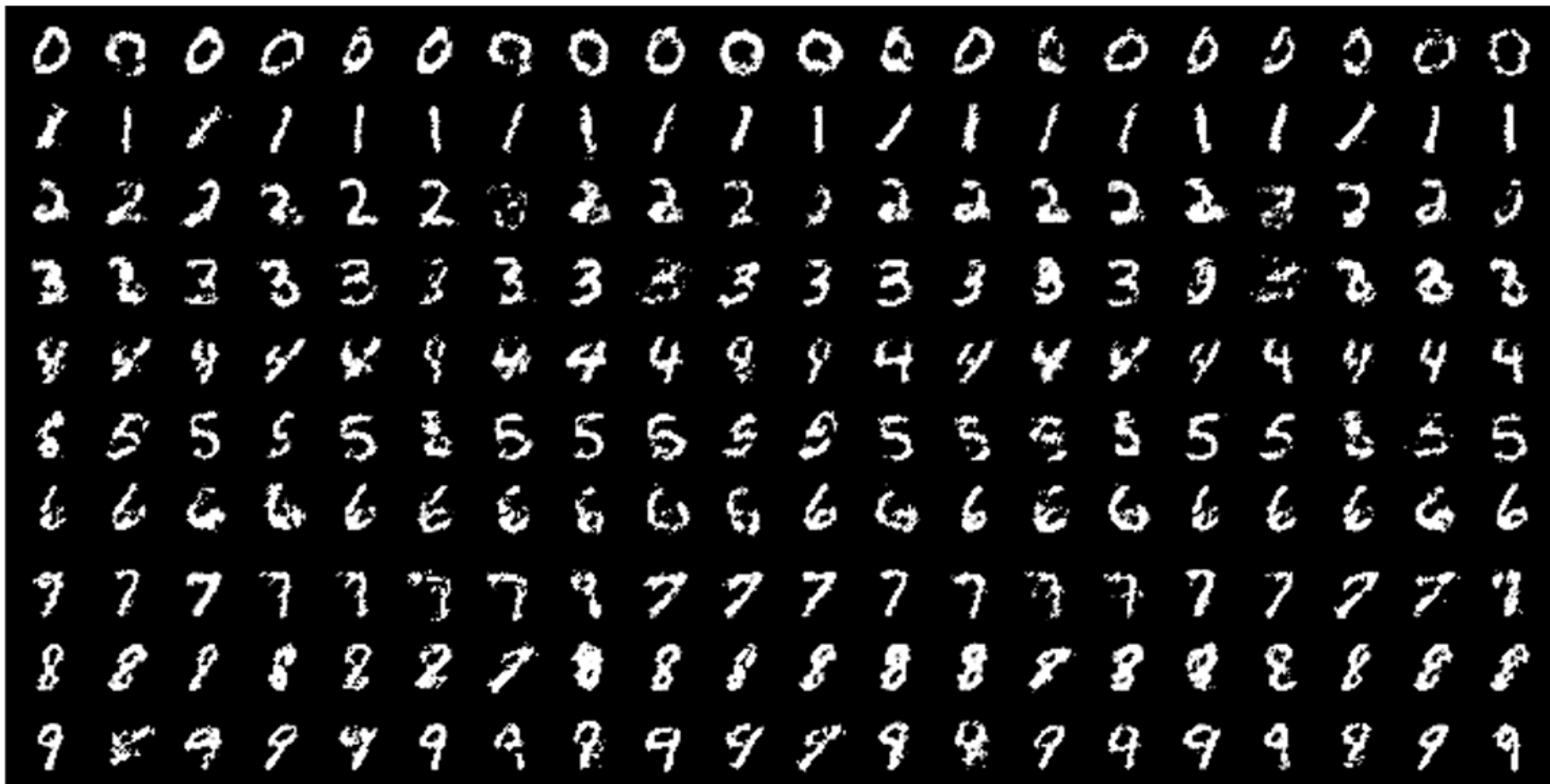


Figure 2: Generated MNIST digits, each row conditioned on one label

Reference:

Antipov, G., Baccouche, M., & Dugelay, J. L. (2017, September). **Face aging with conditional generative adversarial networks**. In *2017 IEEE international conference on image processing (ICIP)* (pp. 2089-2093).

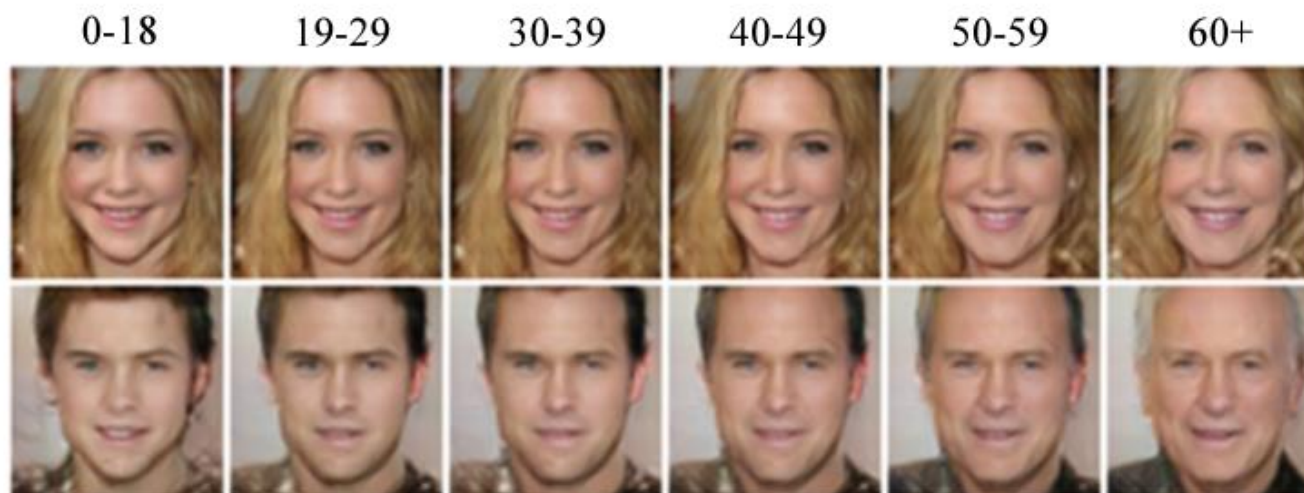


Fig. 2. Examples of synthetic images generated by our Age-cGAN using two random latent vectors z (rows) and conditioned on the respective age categories y (columns).

Reference:

Zhu, J. Y., Park, T., Isola, P., & Efros, A. A. (2017). **Unpaired image-to-image translation using cycle-consistent adversarial networks**. In *Proceedings of the IEEE international conference on computer vision* (pp. 2223-2232).

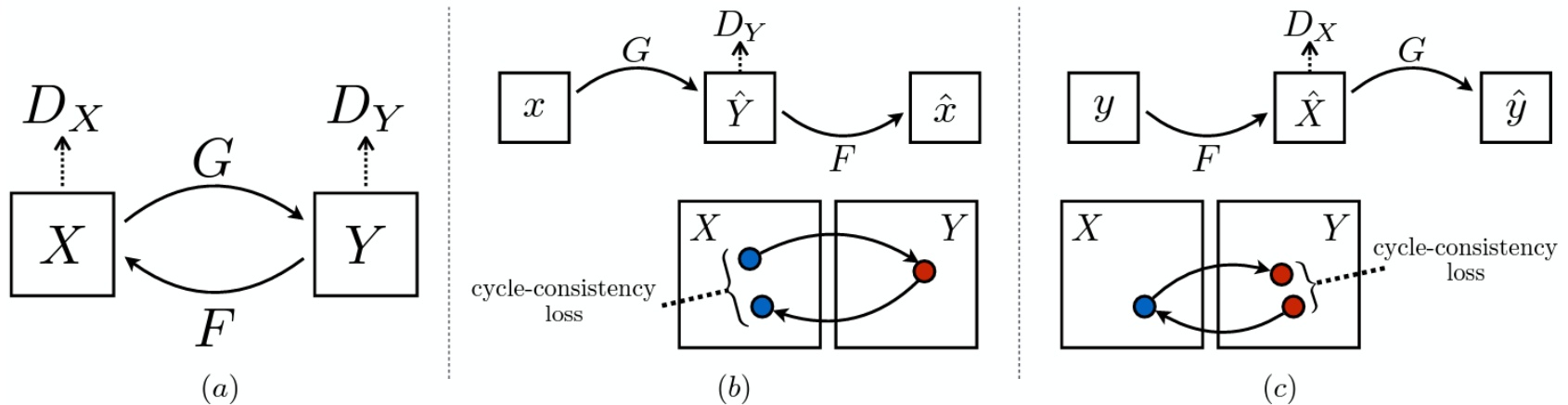


Figure 3: (a) Our model contains two mapping functions $G : X \rightarrow Y$ and $F : Y \rightarrow X$, and associated adversarial discriminators D_Y and D_X . D_Y encourages G to translate X into outputs indistinguishable from domain Y , and vice versa for D_X and F . To further regularize the mappings, we introduce two *cycle consistency losses* that capture the intuition that if we translate from one domain to the other and back again we should arrive at where we started: (b) forward cycle-consistency loss: $x \rightarrow G(x) \rightarrow F(G(x)) \approx x$, and (c) backward cycle-consistency loss: $y \rightarrow F(y) \rightarrow G(F(y)) \approx y$

Cycle GAN



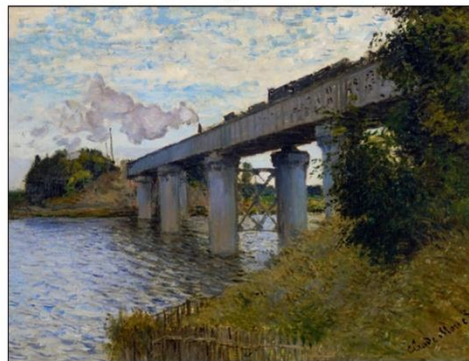
Input



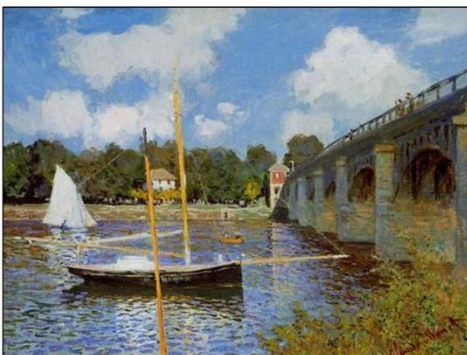
Output



Input

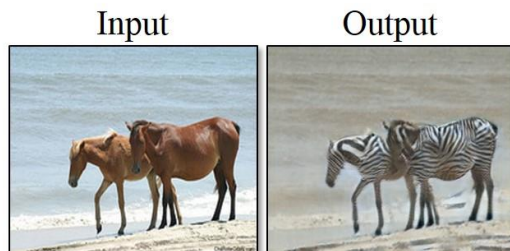


Output



Monet Paintings → Photos

Cycle GAN



horse $\rightarrow$ zebra



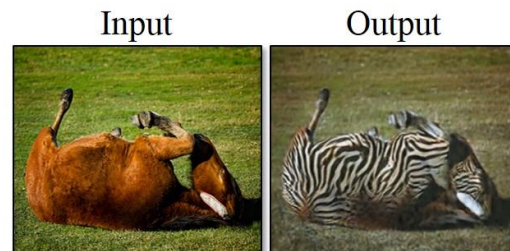
zebra $\rightarrow$ horse



apple $\rightarrow$ orange



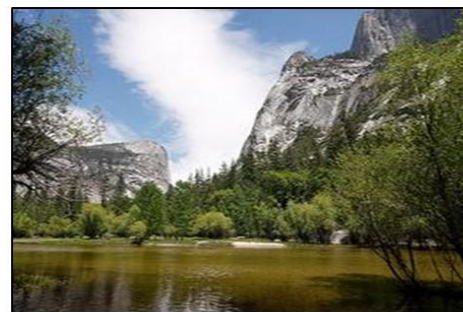
orange $\rightarrow$ apple



Object Transfiguration



winter Yosemite → summer Yosemite

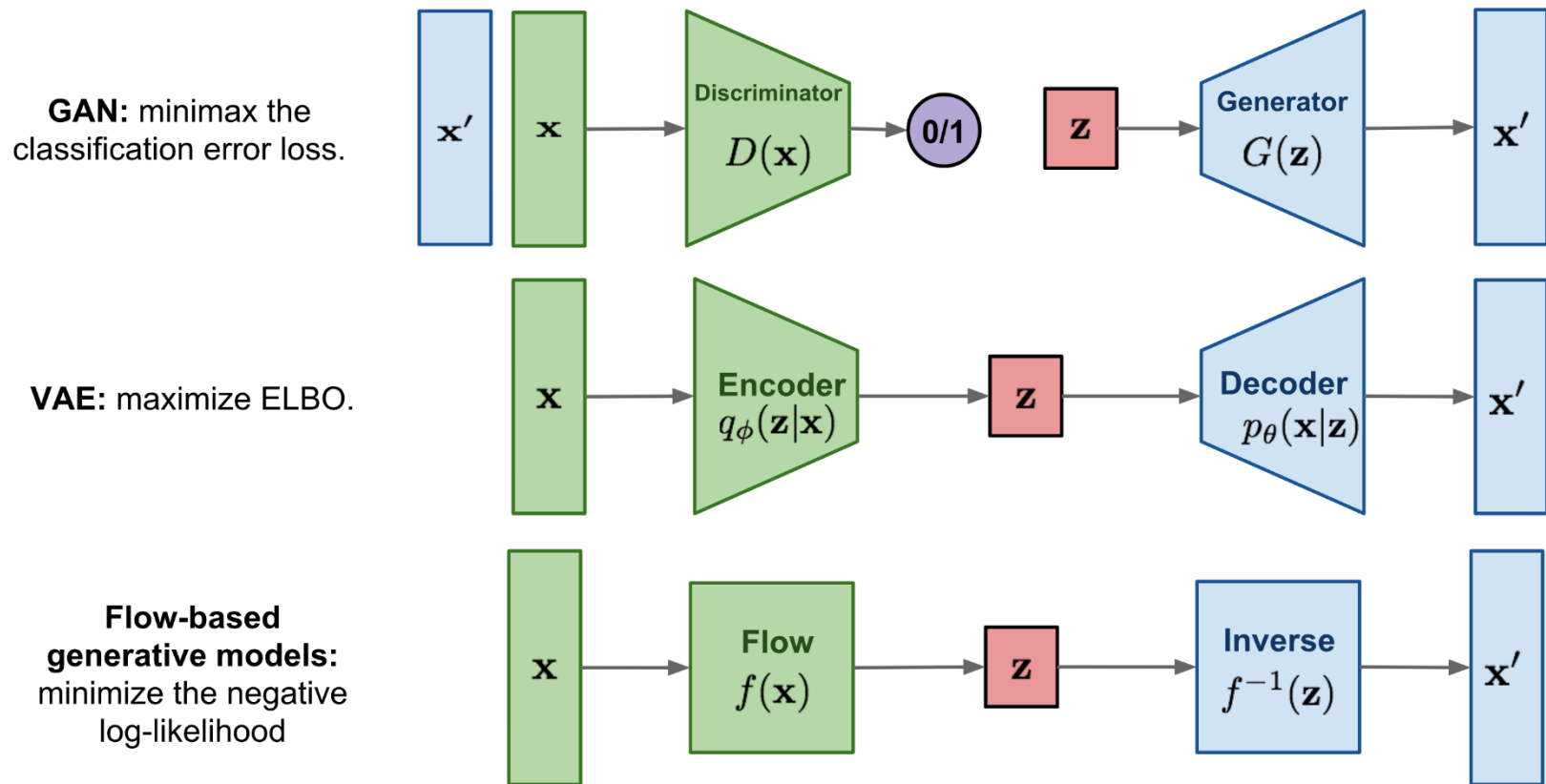


summer Yosemite → winter Yosemite

Season Transfer

Other Generative models

A flow-based generative model is constructed by a sequence of invertible transformations. Unlike GAN and VAE, the **model explicitly learns the data distribution** $p(\mathbf{x})$ and therefore the loss function is simply the negative log-likelihood.



Why invertible transformations ?

$$\mathbf{z} \sim \pi(\mathbf{z}), \mathbf{x} = f(\mathbf{z}), \mathbf{z} = f^{-1}(\mathbf{x})$$
$$p(\mathbf{x}) = \pi(\mathbf{z}) \left| \det \frac{d\mathbf{z}}{d\mathbf{x}} \right| = \pi(f^{-1}(\mathbf{x})) \left| \det \frac{df^{-1}}{d\mathbf{x}} \right|$$

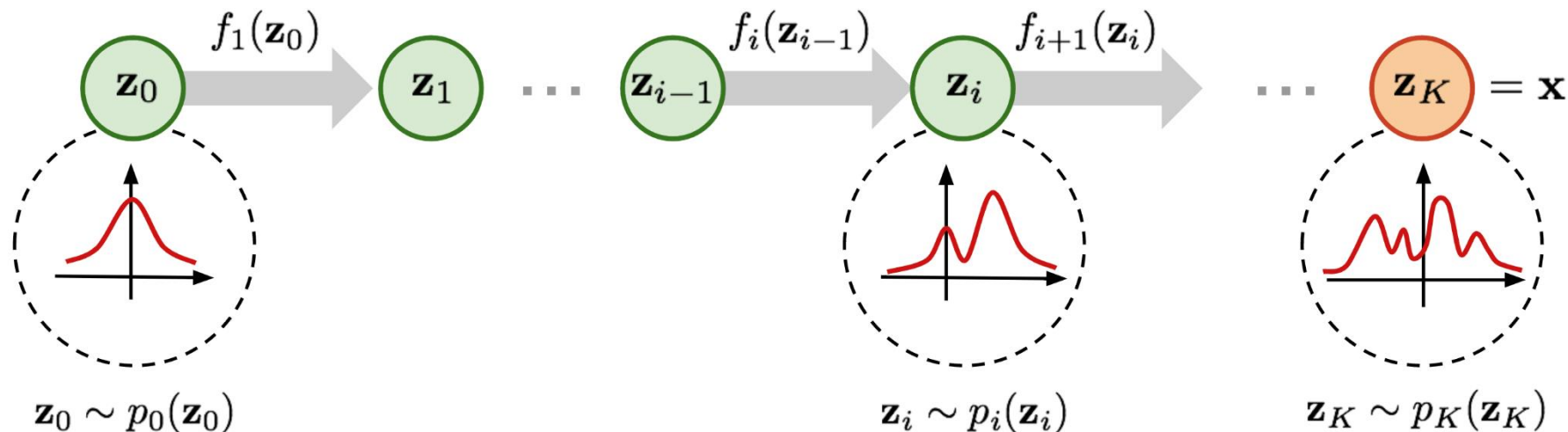
where $\det \frac{\partial f}{\partial \mathbf{z}}$ is the Jacobian determinant of the function f .

Given $\pi(z)$, invertible f , we can explicitly estimate $p(x)$
by maximizing the log-likelihood

Requirement of transformation function f

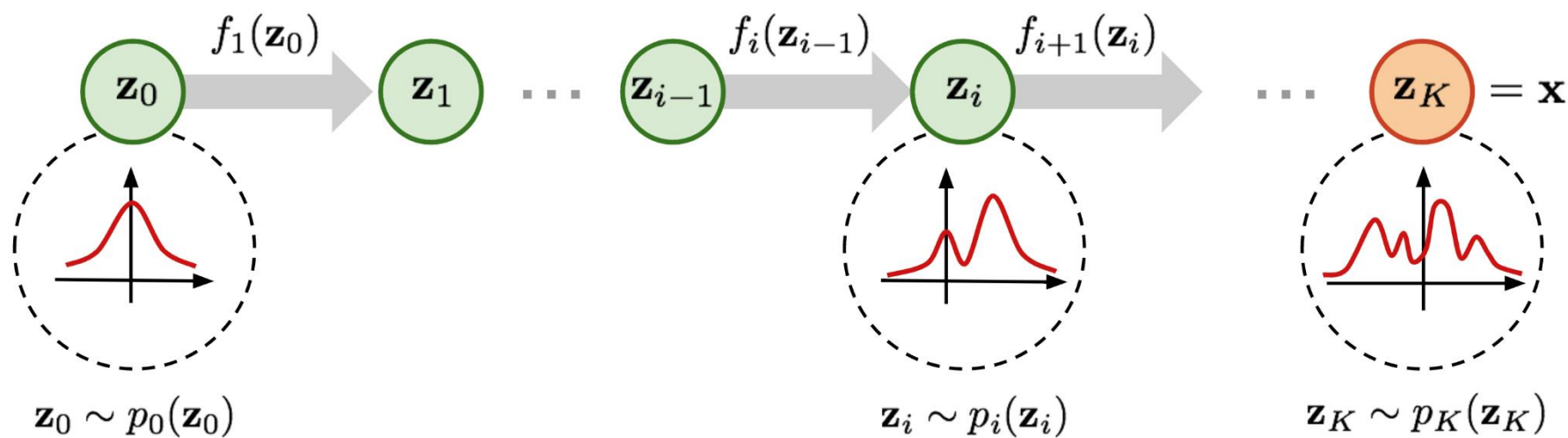
1. It is easily invertible.
2. Its Jacobian determinant is easy to compute.

A normalizing flow transforms a simple distribution into a complex one by applying a sequence of invertible transformation functions.



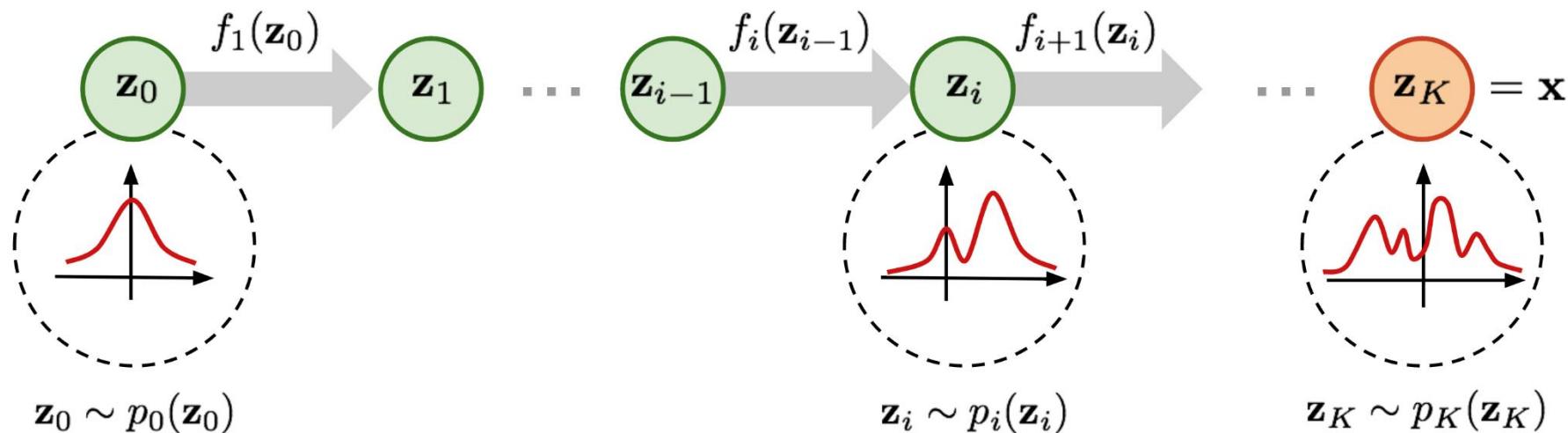
Source: <https://lilianweng.github.io/posts/2018-10-13-flow-models/>

Illustration of a normalizing flow model, transforming a simple distribution $p_0(z_0)$ to a complex one $p_K(z_K)$ step by step.



$$\mathbf{x} = \mathbf{z}_K = f_K \circ f_{K-1} \circ \dots \circ f_1(\mathbf{z}_0)$$

$$\begin{aligned} \log p(\mathbf{x}) &= \log \pi_K(\mathbf{z}_K) = \log \pi_{K-1}(\mathbf{z}_{K-1}) - \log \left| \det \frac{df_K}{d\mathbf{z}_{K-1}} \right| \\ &= \log \pi_{K-2}(\mathbf{z}_{K-2}) - \log \left| \det \frac{df_{K-1}}{d\mathbf{z}_{K-2}} \right| - \log \left| \det \frac{df_K}{d\mathbf{z}_{K-1}} \right| \\ &= \dots \\ &= \log \pi_0(\mathbf{z}_0) - \sum_{i=1}^K \log \left| \det \frac{df_i}{d\mathbf{z}_{i-1}} \right| \end{aligned}$$



As a result, the training criterion of flow-based generative model is simply the negative log-likelihood (NLL) over the training dataset D :

$$\mathcal{L}(\mathcal{D}) = -\frac{1}{|\mathcal{D}|} \sum_{\mathbf{x} \in \mathcal{D}} \log p(\mathbf{x})$$

Methods to implement Normalizing Flow

- **NICE** (Non-linear Independent Component Estimation; [Dinh, et al. 2015](#))
- **RealNVP** (Real-valued Non-Volume Preserving; [Dinh et al., 2017](#))
- **Glow** ([Kingma and Dhariwal, 2018](#))

Suggested reading: [Flow-based Deep Generative Models](#)

Likelihood-based models directly model the probability density function

$$p_{\theta}(\mathbf{x}) = \frac{e^{-f_{\theta}(\mathbf{x})}}{Z_{\theta}}$$

where $f_{\theta}(x) \in \mathbb{R}$ is a real-valued function parameterized by a learnable parameter θ , $Z_{\theta} > 0$ is a normalizing constant dependent on θ , such that $\int p_{\theta}(x) = 1$.

But Z_{θ} is typically an intractable quantity which can not be evaluated.

- The **score function** of a distribution $p(x)$ is defined as

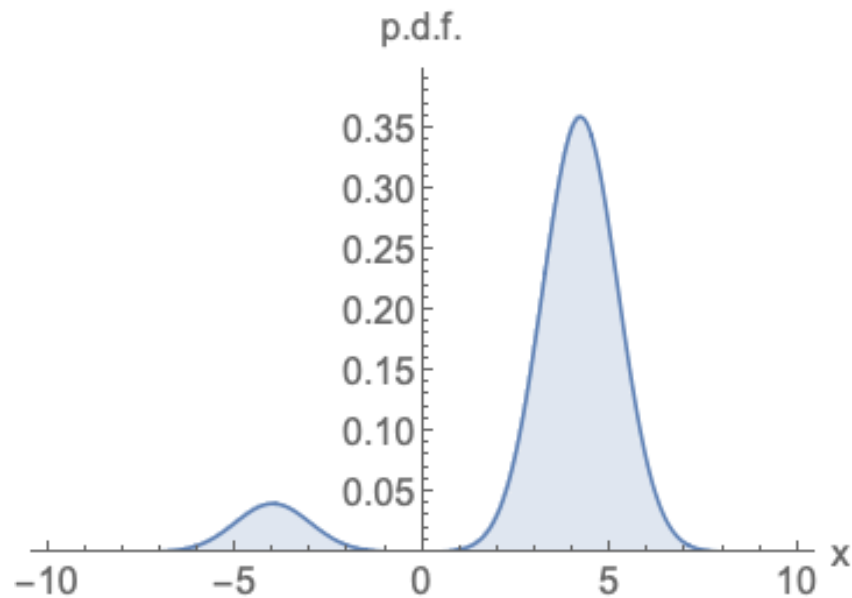
$$\nabla_{\mathbf{x}} \log p(\mathbf{x})$$

and a model for the score function is called a **score-based model**, which we denote as $S_{\theta}(x)$.

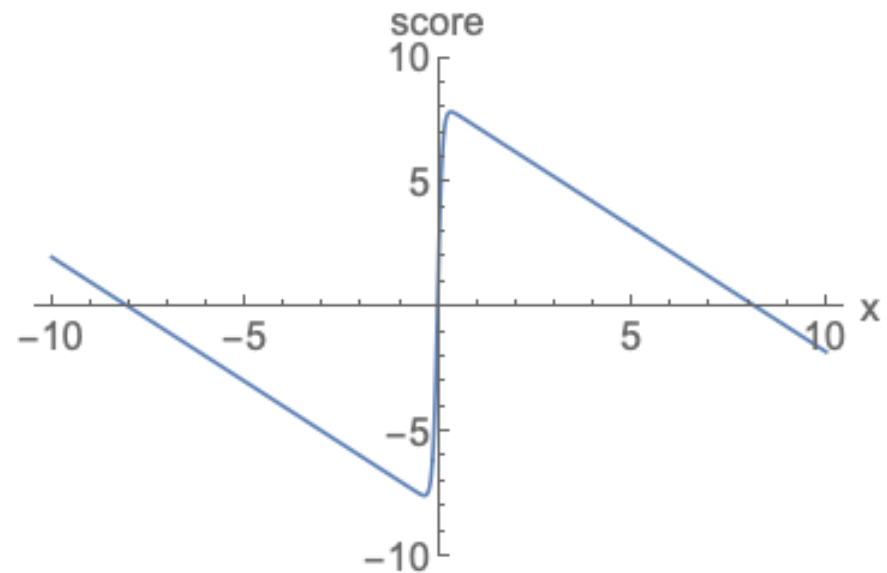
- The score-based model is learned such that $S_{\theta}(x) \approx \nabla_x \log p(x)$.
- By modeling the score function instead of the density function, we can sidestep the difficulty of intractable normalizing constants.

$$\mathbf{s}_{\theta}(\mathbf{x}) = \nabla_{\mathbf{x}} \log p_{\theta}(\mathbf{x}) = -\nabla_{\mathbf{x}} f_{\theta}(\mathbf{x}) - \underbrace{\nabla_{\mathbf{x}} \log Z_{\theta}}_{=0} = -\nabla_{\mathbf{x}} f_{\theta}(\mathbf{x})$$

$$p_{\theta}(\mathbf{x}) = \frac{e^{-f_{\theta}(\mathbf{x})}}{Z_{\theta}}$$



Parameterizing probability density functions. No matter how you change the model family and parameters, it has to be normalized (area under the curve must integrate to one).



Parameterizing score functions. No need to worry about normalization.

- Train score-based models by minimizing the Fisher divergence

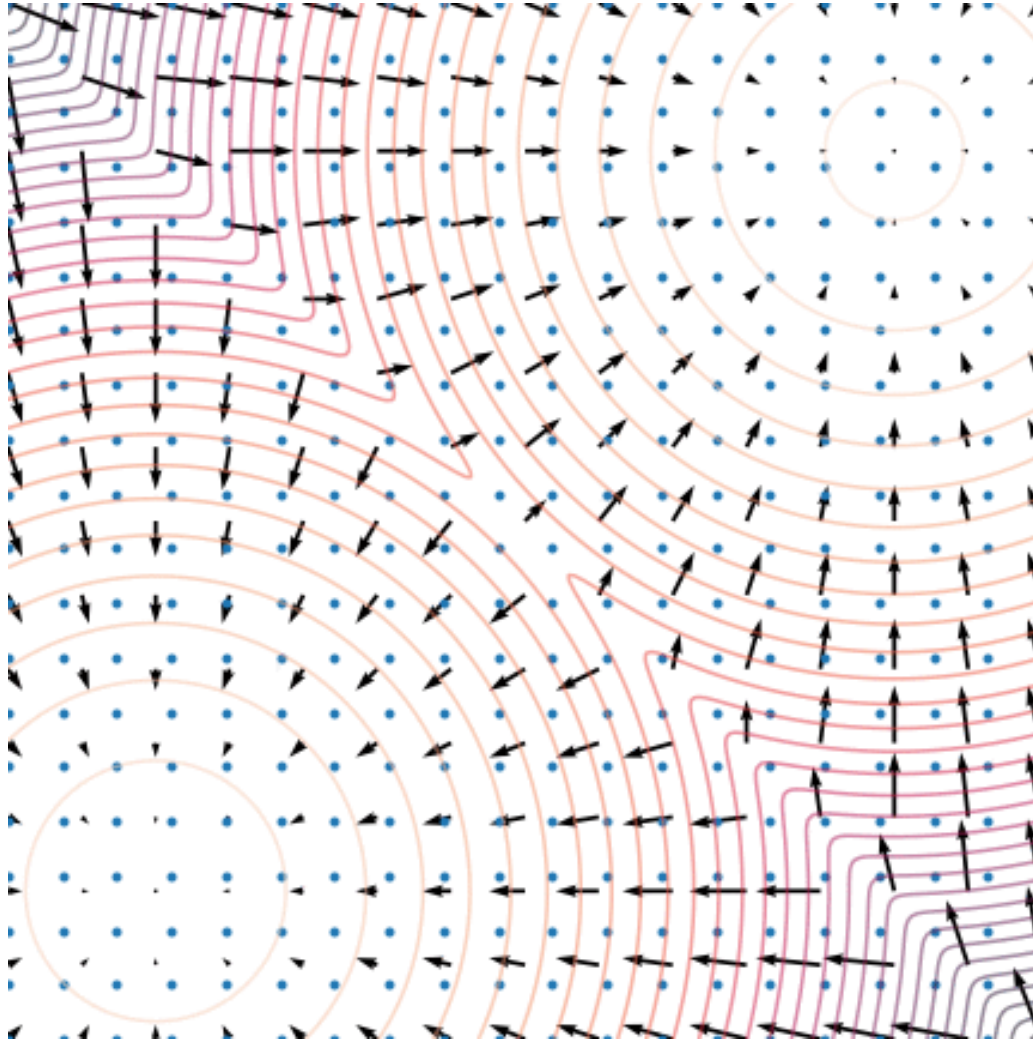
$$\mathbb{E}_{p(\mathbf{x})}[\|\nabla_{\mathbf{x}} \log p(\mathbf{x}) - \mathbf{s}_{\theta}(\mathbf{x})\|_2^2]$$

- Once trained a model $s_{\theta}(x) \approx \nabla_x \log p(x)$, we can use an iterative procedure called Langevin dynamics to draw samples from it.
- It initializes the chain from an arbitrary prior distribution $x_0 \sim \pi(x)$, and then iterates the following

$$\mathbf{x}_{i+1} \leftarrow \mathbf{x}_i + \epsilon \nabla_{\mathbf{x}} \log p(\mathbf{x}) + \sqrt{2\epsilon} \mathbf{z}_i, \quad i = 0, 1, \dots, K,$$

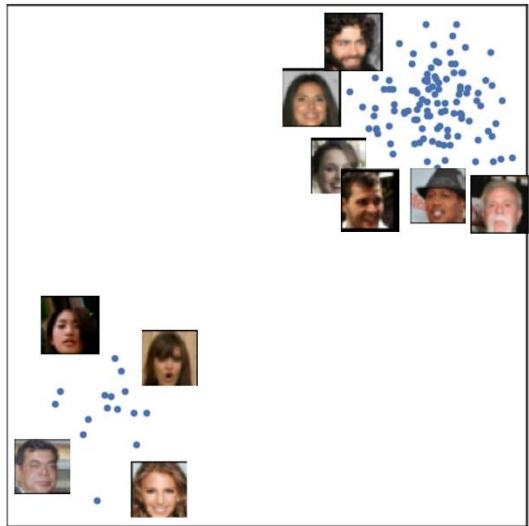
where $\mathbf{z}_i$ follow standard Gaussian distribution.

- When $\epsilon \rightarrow 0$ and $K \rightarrow \infty$, x_K obtained from the procedure converges to a sample from $p(x)$ under some regularity conditions



Source: <https://yang-song.net/blog/2021/score/>

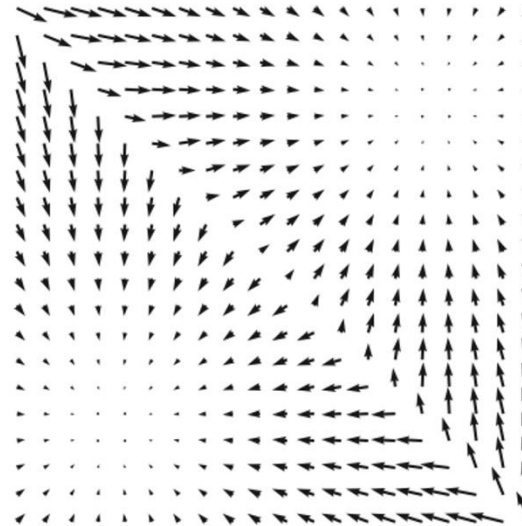
Using Langevin dynamics to sample from a mixture of two Gaussians.



Data samples

$$\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\} \stackrel{\text{i.i.d.}}{\sim} p(\mathbf{x})$$

score
matching



Scores

$$\mathbf{s}_\theta(\mathbf{x}) \approx \nabla_{\mathbf{x}} \log p(\mathbf{x})$$

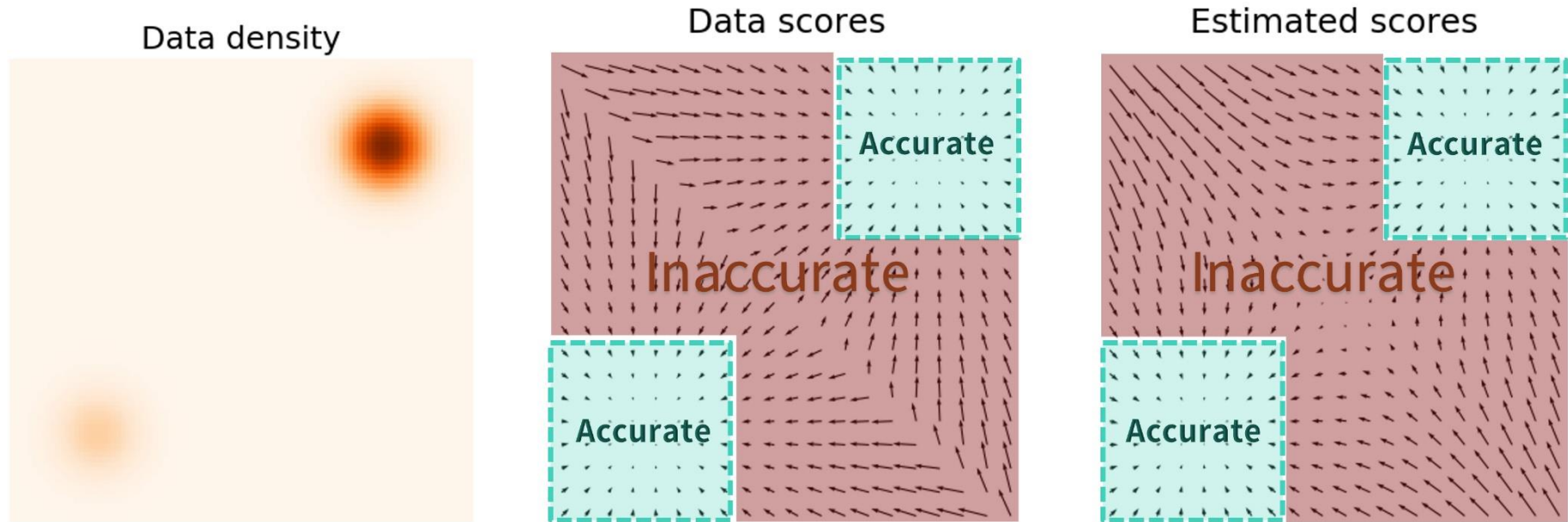
Langevin
dynamics



New samples

Source: <https://yang-song.net/blog/2021/score/>

Score-based generative modeling with
score matching + Langevin dynamics.

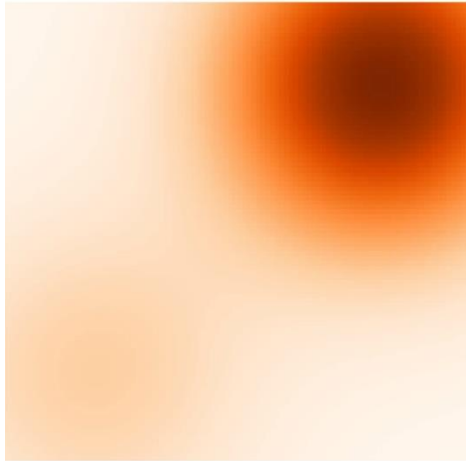


Source: <https://yang-song.net/blog/2021/score/>

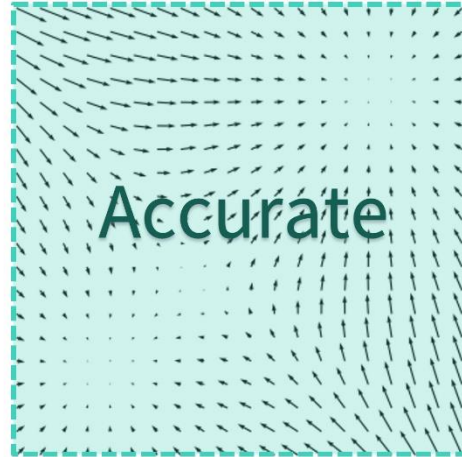
Problems: Estimated scores are only accurate in high density regions.

Score-based models

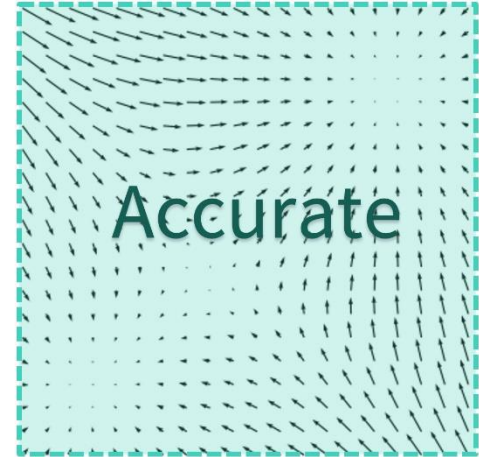
Perturbed density



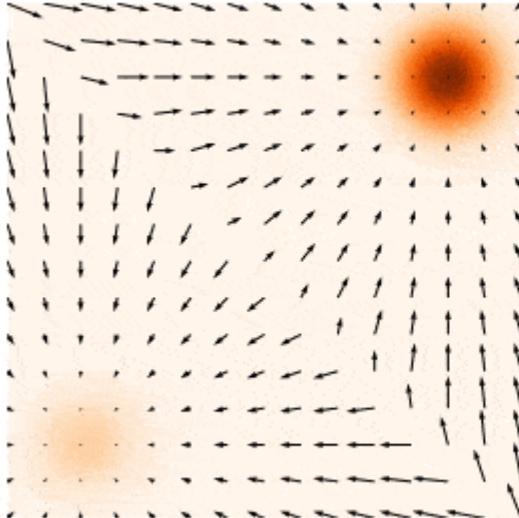
Perturbed scores



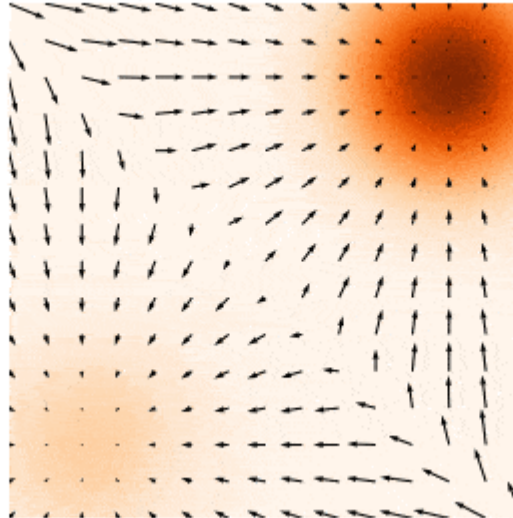
Estimated scores



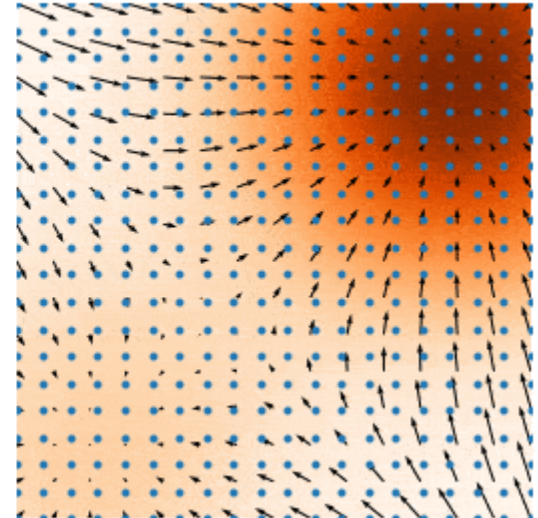
σ_1



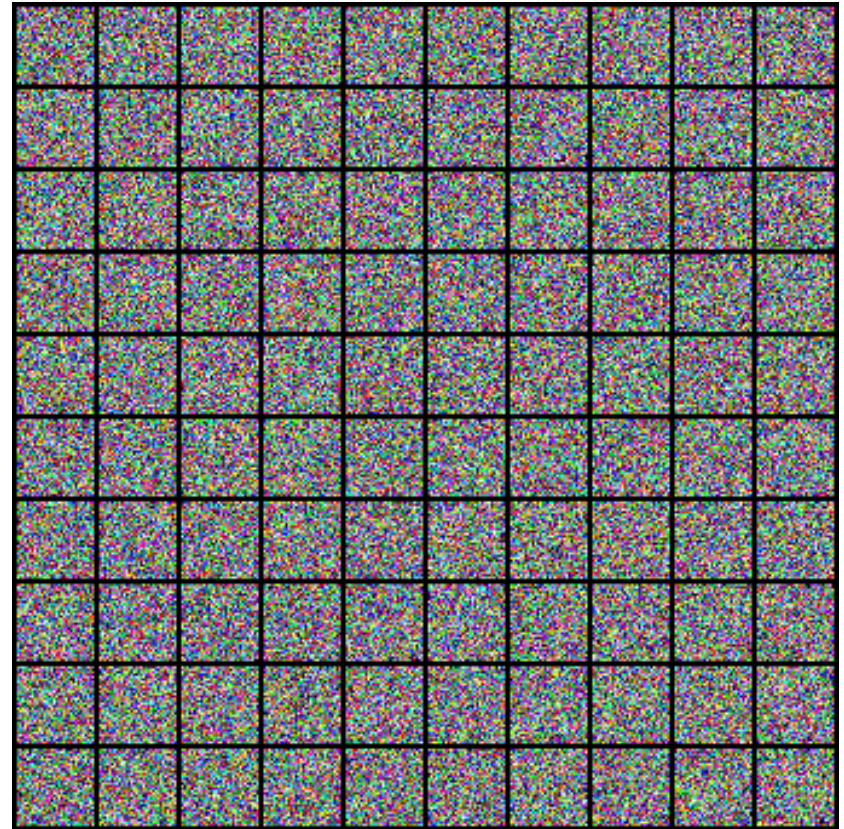
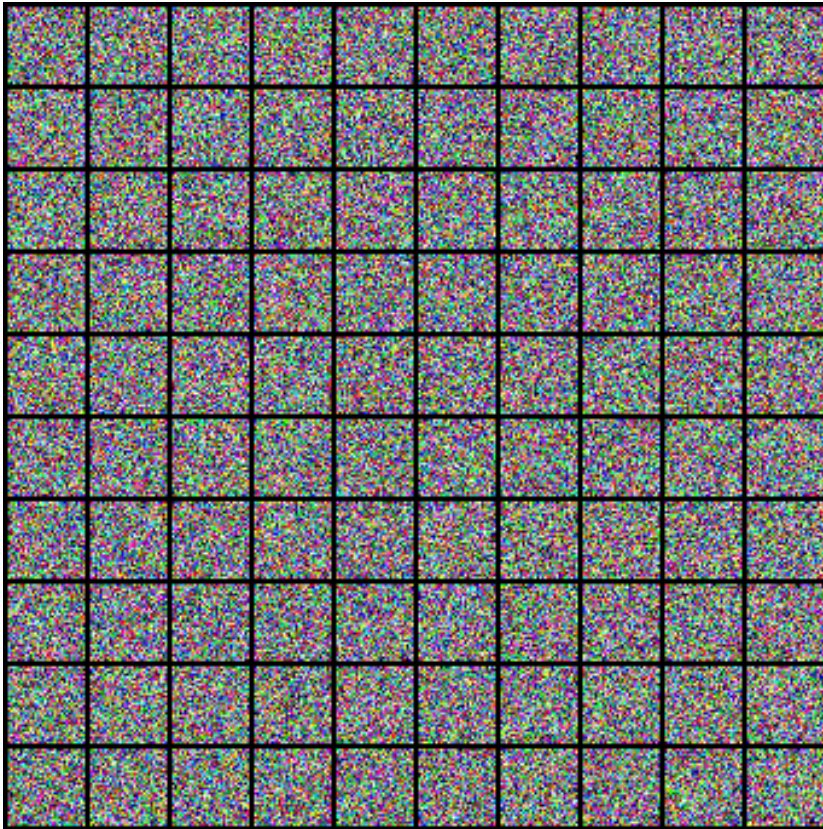
σ_2



σ_3



Solution: perturb data points with noise and train score-based models on the noisy data points instead



Annealed Langevin dynamics for the Noise Conditional Score Network (NCSN) model trained on CelebA (left) and CIFAR-10 (right).



1024 x 1024 samples from a score-based model
trained on the FFHQ dataset.



256 x 256 samples on LSUN bedroom.



256 x 256 samples on CelebA-HQ.

Suggested reading:

- [Generative Modeling by Estimating Gradients of the Data Distribution](#)
- [What are Diffusion Models?](#)